



VIT[®]

BHOPAL

ROOMMATE / ROOM SWAPPING SYSTEM

Project Type: Console-Based Java Application

Student Name: DHRITI SHUKLA

Registration Number: 25BAI10259

Faculty Name: KAMLESH CHANDRAVANSHI

Academic Year: 2026–27

2. Introduction

The **Roommate / Room Swapping System** is a Java-based console application developed to simplify the management of student rooms and room-swapping requests.

In a college hostel, students may sometimes need to exchange their rooms with other students. Managing these requests manually can be time-consuming and can lead to confusion while keeping track of students, rooms, and pending requests.

This project provides a simple digital system where students can be added and searched, room information can be viewed, and students can create room-swapping requests. An administrator can then process these requests by accepting or rejecting them.

The project is completely **console-based**, meaning that it can be executed directly through the command line without requiring a graphical interface.

The project also demonstrates important Java and Object-Oriented Programming concepts such as classes, objects, encapsulation, constructors, methods, ArrayList, conditional statements, loops, and input validation.

3. Problem Statement

Managing hostel rooms and room-swapping requests manually can be difficult when the number of students increases.

Students may have difficulty finding information about other students and their rooms. Administrators also need to keep

track of multiple swap requests and make sure that requests are processed correctly.

The main problems include:

- Difficulty in maintaining student and room information.
- Difficulty in searching for students.
- Manual handling of room swap requests.
- Possibility of confusion while tracking request status.
- Possibility of invalid or duplicate operations.
- Difficulty in updating room information after a successful swap.

Therefore, there is a need for a simple system that can organize student and room information and provide a structured process for requesting and processing room swaps.

4. Functional Requirements

The system provides the following functional requirements:

4.1 Add Student

The system allows the user to add a new student by entering:

- Student ID
- Student Name
- Email
- Current Room

The student information is then stored in the system.

4.2 View All Students

The system displays all registered students along with their relevant information.

4.3 Search Student

The user can search for a student using the Student ID.

If the student exists, their information is displayed. Otherwise, an appropriate error message is shown.

4.4 View Rooms

The system displays available and occupied rooms along with:

- Room number
- Room type
- Occupancy status

4.5 Request Room Swap

A student can request a room swap with another student by entering their Student IDs.

The system checks whether both students exist and whether the request is valid.

4.6 View Swap Requests

The system displays all room swap requests along with:

- Request ID
- First student

- Second student
- Request status

4.7 Process Swap Request

An administrator can process a pending swap request.

The administrator can:

- Accept the request
- Reject the request

4.8 Update Room Information

When a swap request is accepted, the rooms of the two students are exchanged automatically.

4.9 Input Validation

The system validates user input and prevents invalid operations such as:

- Invalid Student ID
- Invalid Request ID
- Invalid menu option
- Student swapping with themselves
- Processing an already processed request

5. Non-functional Requirements

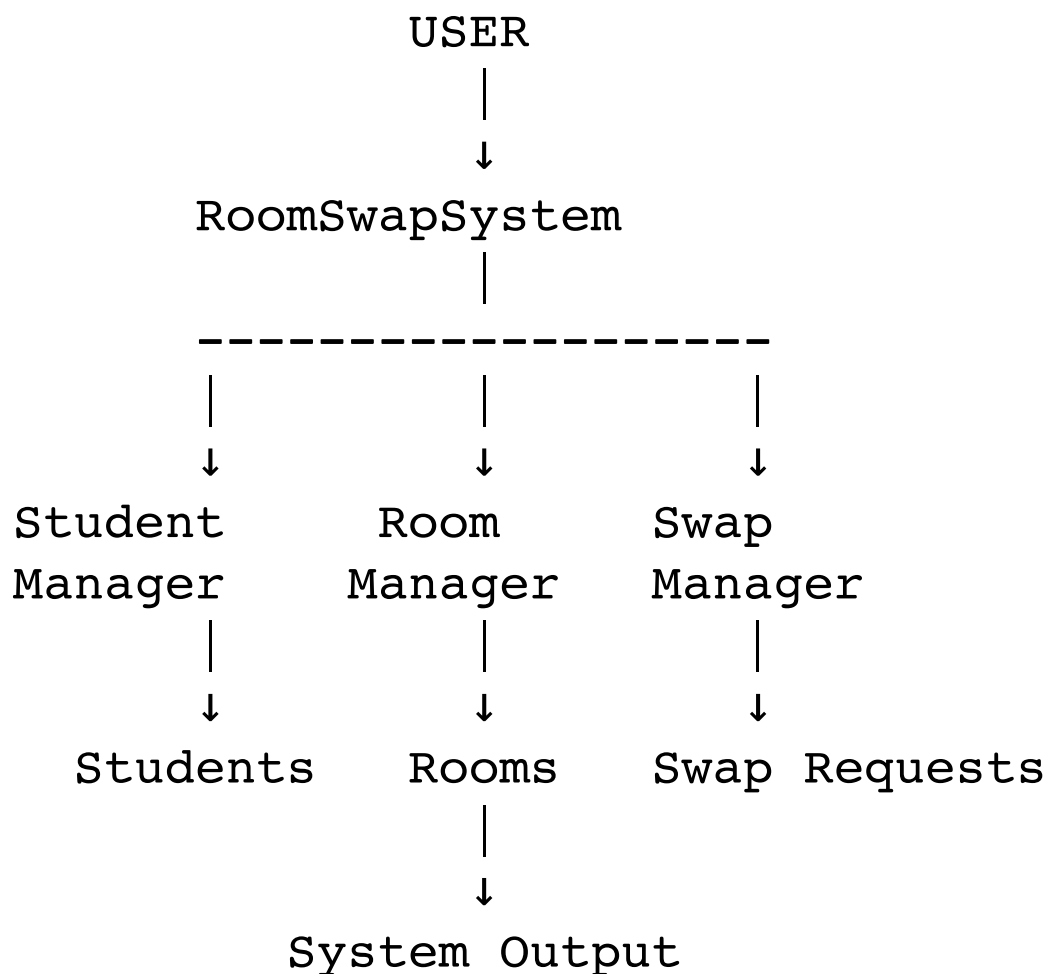
The project also satisfies the following non-functional requirements.

Requireme	Description
Usability	The system provides a simple menu-driven interface that is easy to understand.
Performanc e	Operations such as searching and displaying information are performed quickly for the expected project size.
Reliability	The application handles invalid inputs without unnecessarily terminating the program.
Maintainab ility	The project is divided into multiple classes, making the code easier to understand and modify.
Error Handling	Clear error messages are displayed when invalid operations are performed.
Scalability	The modular structure allows future features such as database connectivity to be
Portability	The application can run on systems that have Java installed and does not require a

6. System Architecture

The project follows a simple modular architecture.

Architecture Flow



Components

1. RoomSwapSystem

This is the main class of the application. It provides the menu, accepts user input, and connects the different modules.

2. StudentManager

This module manages student-related operations such as adding, searching, and displaying students.

3. RoomManager

This module manages room-related operations such as viewing rooms and checking room information.

4. SwapManager

This module manages room swap requests and handles accepting or rejecting requests.

5. Model Classes

The model classes represent the main entities:

- Student
- Room
- SwapRequest
- Admin

6. Data Storage

The current project uses Java `ArrayList` to temporarily store students, rooms, and swap requests.

7. Design Diagrams

7.1 Use Case Diagram

The main actors in the system are:

Student

The student can:

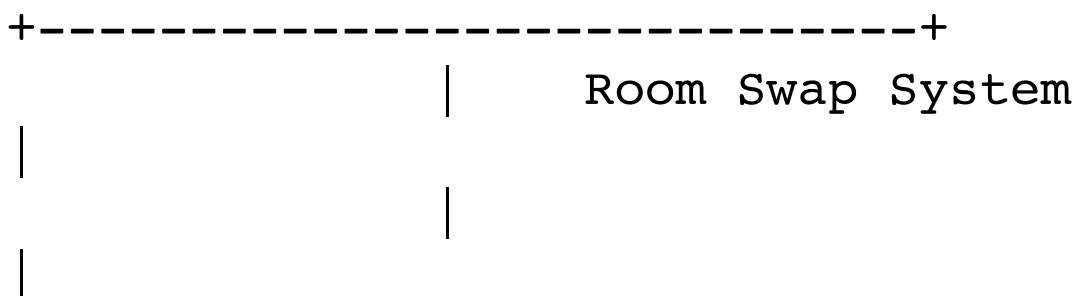
- View students
- Search students
- View rooms
- Create room swap requests
- View swap request information

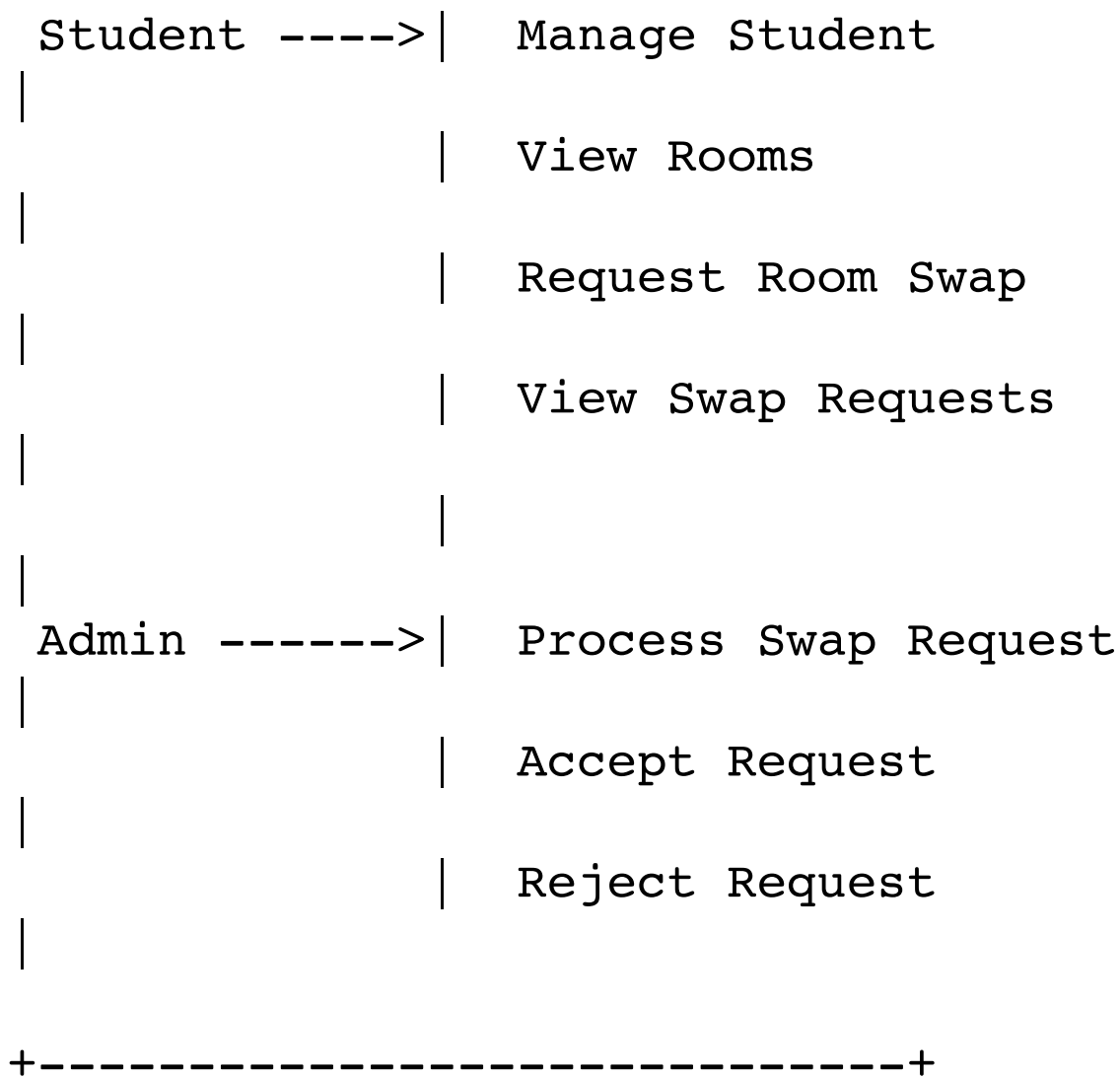
Admin

The administrator can:

- View swap requests
- Accept requests
- Reject requests
- Process room swaps

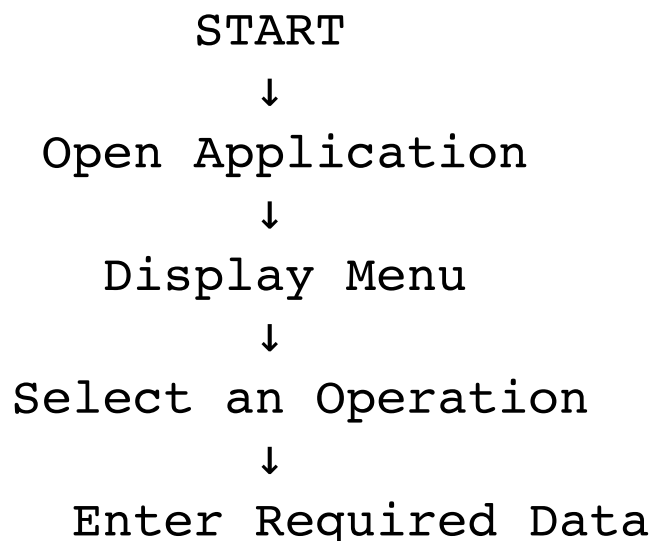
Use Case Representation

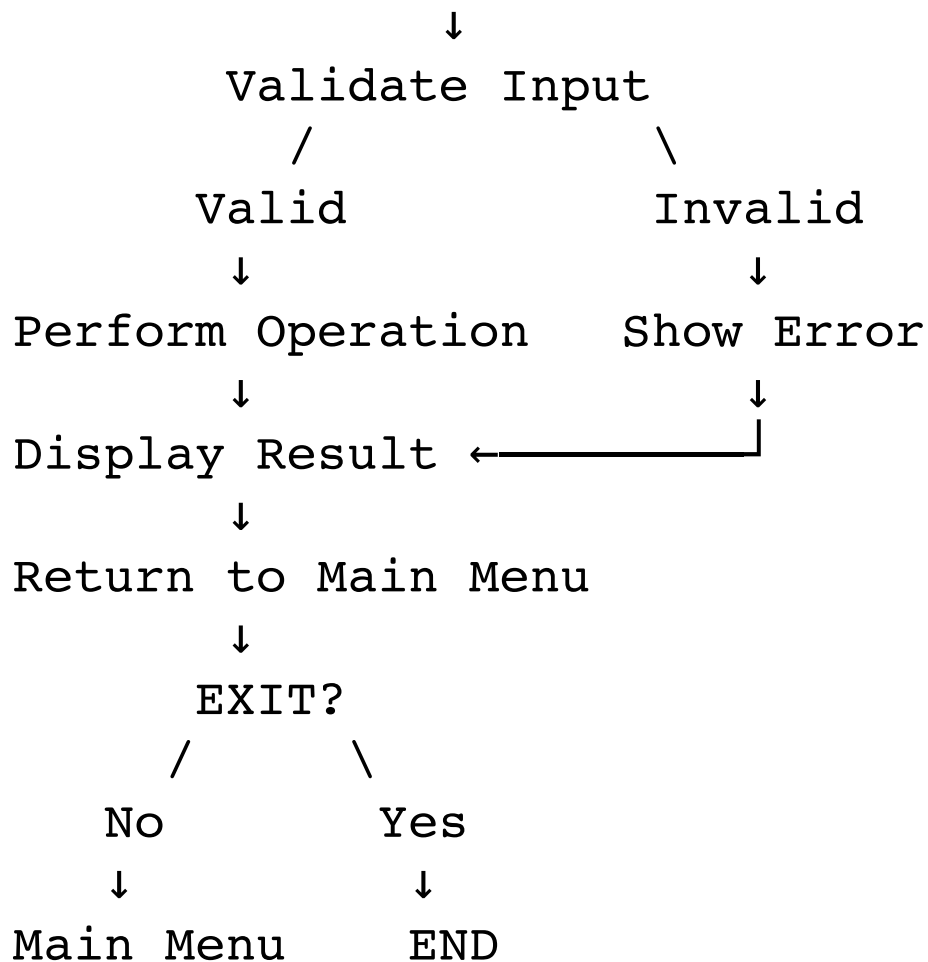




7.2 Workflow Diagram

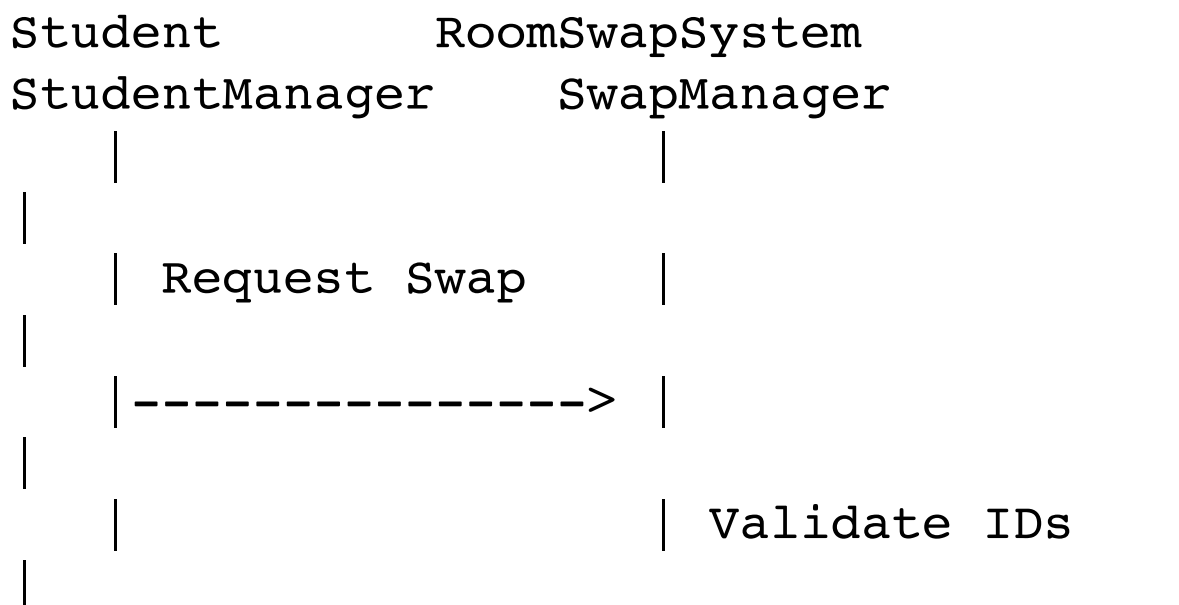
The general workflow of the system is:

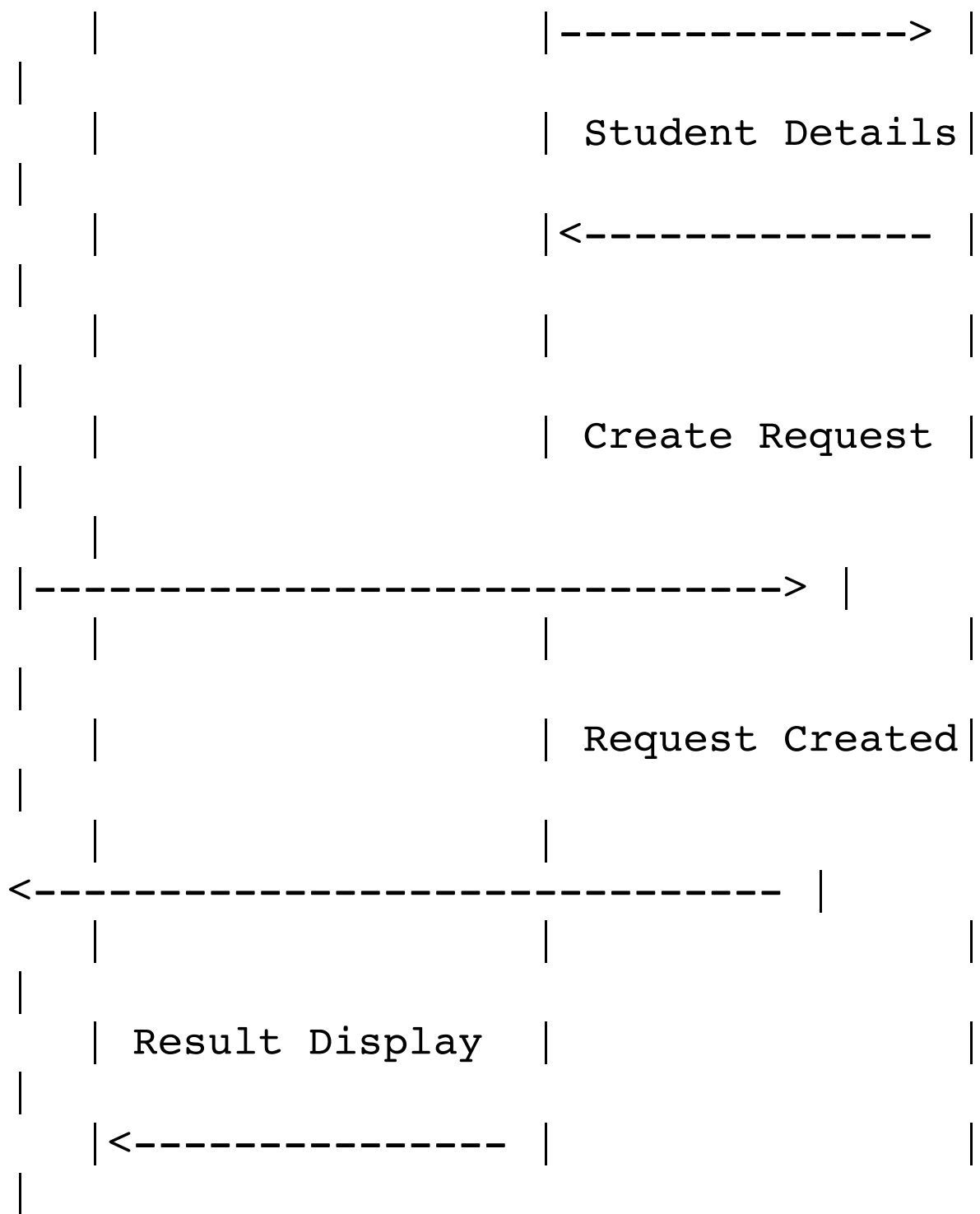




7.3 Sequence Diagram

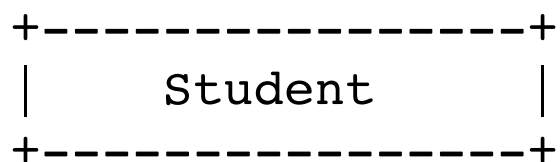
The following sequence represents the process of creating a room swap request:





7.4 Class / Component Diagram

The project contains eight main classes.



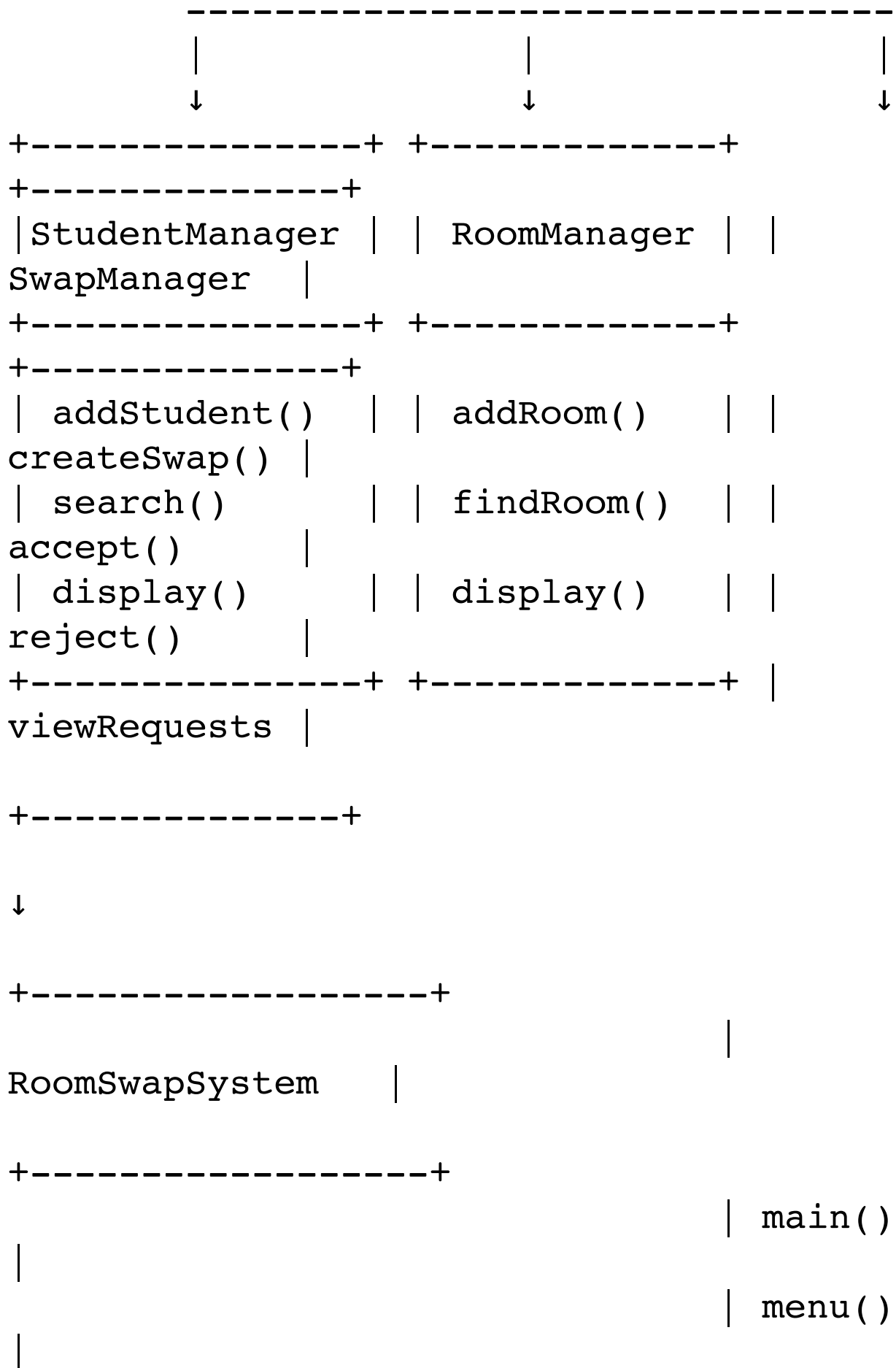
studentId	
name	
email	
currentRoom	
+-----+	

+-----+	
Room	
+-----+	
roomNumber	
roomType	
occupied	
+-----+	

+-----+	
	SwapRequest
+-----+	
	requestId
	student1
	student2
	status
+-----+	

+-----+	
Admin	
+-----+	
adminId	
name	
+-----+	

↓



handling		input
----------	--	-------

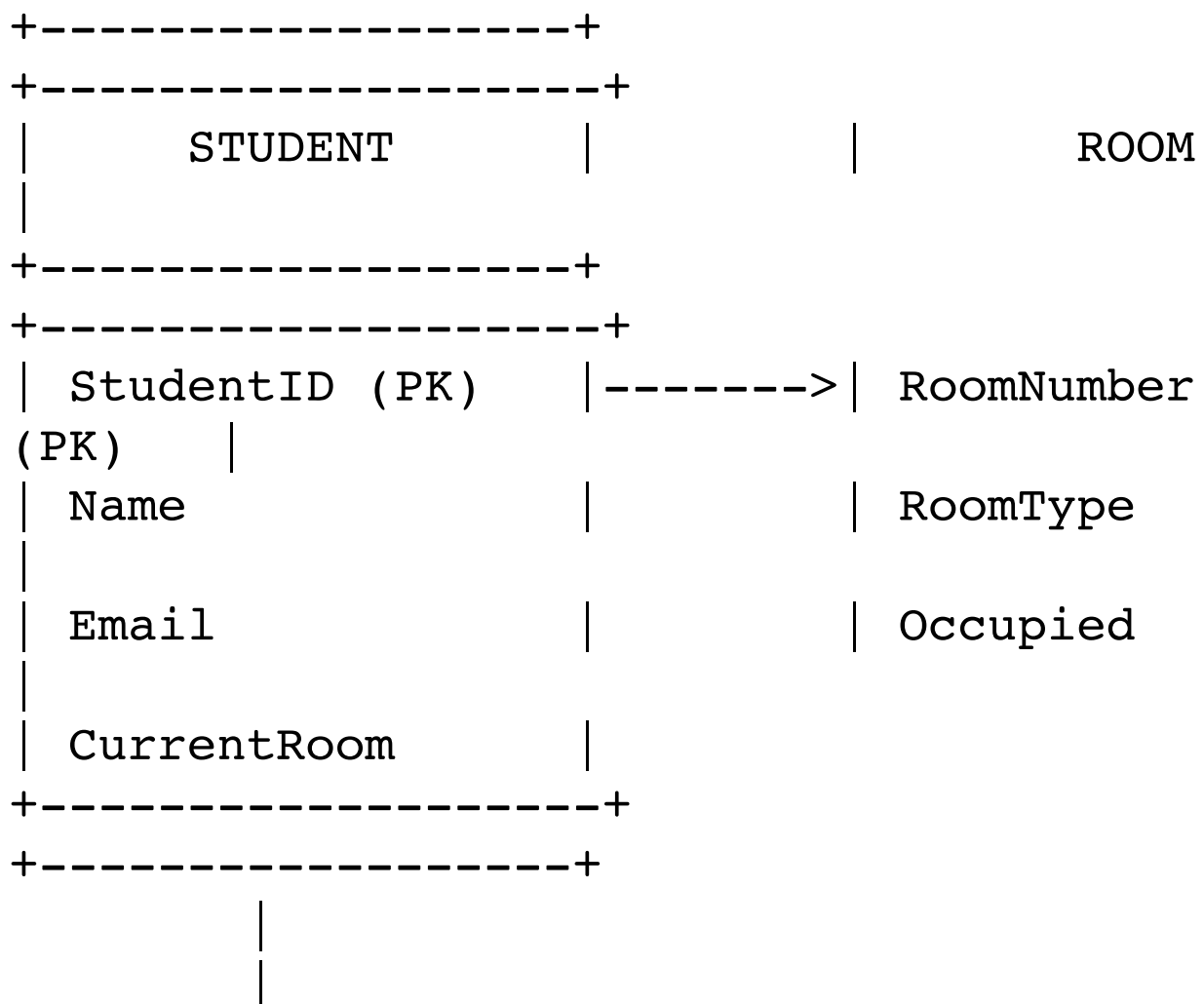
+-----+

7.5 ER Diagram

The current version of the project does **not use a database**. It stores information temporarily using Java ArrayList.

Therefore, an ER diagram is **not required for the actual implementation**.

However, a conceptual representation of the entities for a future database version can be shown as:



↓

+-----+	
	SWAP_REQUEST
+-----+	
	RequestID (PK)
	Student1
	Student2
	Status
+-----+	

This structure can be converted into actual database tables in a future version.

8. Design Decisions & Rationale

Several design decisions were made while developing the project.

8.1 Java

Java was selected because it is suitable for implementing Object-Oriented Programming concepts such as classes, objects, encapsulation, and methods.

8.2 Console Interface

A console-based interface was selected because it keeps the application simple and allows the complete project to run directly from the command line.

8.3 Multiple Classes

The project is divided into eight classes instead of putting all functionality into one class.

This makes the project more modular and easier to understand.

8.4 Manager Classes

Separate manager classes were created for students, rooms, and swap requests.

This prevents the main class from becoming too large and keeps related operations together.

8.5 ArrayList

`ArrayList` was selected for storing students, rooms, and swap requests because it provides a simple way to store multiple objects dynamically.

8.6 No Database

A database was not included in the current version because the main focus of the project is Java programming and Object-Oriented Programming.

The project can be extended with database connectivity in the future.

8.7 Input Validation

Validation was included to prevent common errors and make the application more reliable.

9. Implementation Details

The project consists of the following classes:

Class	Responsibility
<code>Student.java</code>	Stores student details such as ID, name, email, and current room.

Room.java	Stores room number, room type, and occupancy status.
SwapRequest.java	Stores swap request details and request status.
Admin.java	Represents the administrator.
StudentManager.java	Handles student-related operations.
RoomManager.java	Handles room-related operations.
SwapManager.java	Handles room swap requests.
RoomSwapSystem.java	Contains the main method, menu, and overall program flow.

Main Menu

The application provides the following options:

===== ROOM SWAPPING SYSTEM =====

1. Add Student
2. View All Students
3. Search Student
4. View Rooms
5. Request Room Swap
6. View Swap Requests
7. Process Swap Request
8. Exit

Compilation

The Java files can be compiled using:

```
javac src/*.java
```

Execution

The application can be started using:

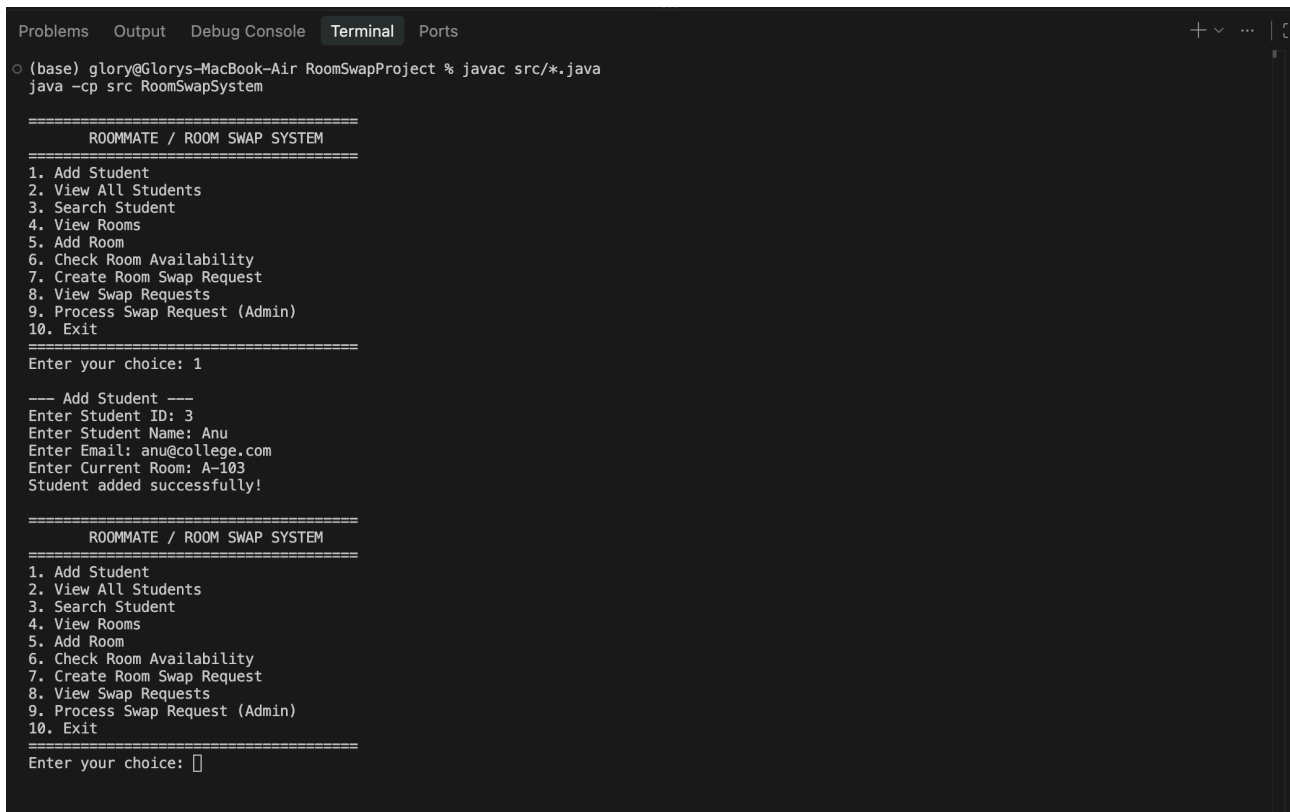
```
java -cp src RoomSwapSystem
```

OOP Concepts Implemented

The project demonstrates:

- Classes and objects
- Encapsulation
- Constructors
- Getters and setters
- Methods
- Object relationships
- ArrayList
- Conditional statements
- Loops
- Switch statements
- Input validation
- Error handling

10. Screenshots / Results



```
Problems Output Debug Console Terminal Ports
(base) glory@Glorys-MacBook-Air RoomSwapProject % javac src/*.java
java -cp src RoomSwapSystem

=====
ROOMMATE / ROOM SWAP SYSTEM
=====
1. Add Student
2. View All Students
3. Search Student
4. View Rooms
5. Add Room
6. Check Room Availability
7. Create Room Swap Request
8. View Swap Requests
9. Process Swap Request (Admin)
10. Exit
=====
Enter your choice: 1

--- Add Student ---
Enter Student ID: 3
Enter Student Name: Anu
Enter Email: anu@college.com
Enter Current Room: A-103
Student added successfully!

=====
ROOMMATE / ROOM SWAP SYSTEM
=====
1. Add Student
2. View All Students
3. Search Student
4. View Rooms
5. Add Room
6. Check Room Availability
7. Create Room Swap Request
8. View Swap Requests
9. Process Swap Request (Admin)
10. Exit
=====
Enter your choice: 
```

11. Testing Approach

Testing was performed using different valid and invalid inputs.

The main purpose of testing was to verify that each feature works correctly and that the system handles incorrect input without crashing.

Test ID	Test Case	Expected Result
---------	-----------	-----------------

T01	Add a valid student	Student is added successfully.
T02	Search for an existing student	Correct student information is displayed.
T03	Search for a non-existing student	Appropriate error message is displayed.
T04	View rooms	Room details are displayed correctly.
T05	Create a valid swap request	Swap request is created successfully.
T06	Student tries to swap with themselves	System rejects the request.
T07	Accept a pending request	Request is accepted and rooms are exchanged.
T08	Reject a pending request	Request status changes to Rejected.
T09	Process an already processed request	System prevents the operation.
T10	Enter an invalid menu option	Appropriate error message is displayed.

Testing Method

The project mainly uses **functional testing**.

Each menu option was tested individually using both valid and invalid inputs.

The output displayed by the system was then compared with the expected result.

12. Challenges Faced

Several challenges were faced during the development of the project.

12.1 Managing Multiple Classes

Dividing the project into multiple classes while keeping the code understandable was initially challenging.

12.2 Managing Student and Room Relationships

The system needs to correctly connect each student with their current room.

12.3 Room Exchange Logic

When a request is accepted, the rooms of two students need to be exchanged correctly without losing the original room information.

12.4 Input Validation

Different invalid inputs had to be handled, such as incorrect Student IDs, incorrect Request IDs, and invalid menu options.

12.5 Preventing Invalid Swaps

The system needed to prevent a student from creating a room swap request with themselves.

12.6 Request Status Management

Once a request is accepted or rejected, it should not be processed again.

13. Learnings & Key Takeaways

This project helped in understanding both Java programming and basic software design.

The major learnings include:

- Learned how to create and use classes and objects.
- Improved understanding of Object-Oriented Programming.
- Learned how encapsulation is implemented using private variables and getters/setters.

- Learned how constructors initialize objects.
- Learned how to use `ArrayList` to store multiple objects.
- Learned how different classes can communicate with each other.
- Learned how to divide a large problem into smaller modules.
- Learned how to create a menu-driven console application.
- Improved understanding of input validation.
- Learned how to handle invalid user operations.
- Understood the importance of testing different types of inputs.
- Learned how UML diagrams can be used to plan a software system before implementation.

14. Future Enhancements

The current system can be improved with several additional features.

14.1 Database Integration

A database such as MySQL can be added to permanently store student, room, and swap-request information.

14.2 File Handling

File handling can be used to save information so that data is not lost when the program is closed.

14.3 Login System

Separate login systems can be created for students and administrators.

14.4 Roommate Preference Matching

Students could enter preferences such as study habits, course, or other requirements, and the system could help find suitable roommates.

14.5 Advanced Search

Students could search rooms based on:

- Room type
- Availability
- Current occupants
- Other preferences

14.6 Notifications

The system could notify students when their swap request is accepted or rejected.

14.7 Admin Dashboard

An administrator dashboard could provide information about:

- Total students
- Occupied rooms
- Available rooms
- Pending requests

- Completed swaps

14.8 GUI / Web Application

The console application could later be converted into a graphical or web-based application.

15. References

1. Oracle Java Documentation – Java Programming Language and Standard Library documentation.
2. Oracle Java Tutorials – Classes, Objects, Constructors, Encapsulation, and Collections.
3. Java `ArrayList` Documentation – Used for managing collections of objects.
4. Java `Scanner` Documentation – Used for taking input from the command line.
5. Java Development Kit (JDK) Documentation.
6. Course learning materials and classroom notes related to Java and Object-Oriented Programming.
7. Course learning materials related to software design, system architecture, UML diagrams, and testing.